

/goal et dix commandes natives pour faire avancer un projet avec un agent IA

MULTI-OUTILS · 18 MIN DE LECTURE

Un objectif vérifiable, dix commandes natives, et des consignes prêtes à adapter pour travailler avec les agents en ligne de commande type Codex ou Claude Code.

Sommaire

- Introduction
- 00 · /goal : donner un objectif et une preuve de réussite
- 01 · /plan : décider du chemin
- 02 · /init : écrire les règles du projet
- 03 · /model : choisir son moteur
- 04 · /permissions : donner un périmètre
- 05 · /mcp : vérifier les connexions
- 06 · /status : lire les instruments
- 07 · /review : faire relire le travail
- 08 · /compact : alléger le contexte
- 09 · /resume : retrouver le fil
- 10 · /clear : ouvrir un nouveau départ
- Un projet, une session
- Références

Introduction

Vous savez ce que vous voulez obtenir. Vous n'avez pas envie de dicter chaque clic, chaque correction et chaque nouveau contrôle. Avec un objectif bien formulé, vous pouvez confier une suite de travail à un agent et juger le résultat sur des preuves concrètes, pas sur une impression.

Commencez par `/goal`, puis découvrez dix autres commandes natives. Pour chacune : le bon moment pour l'utiliser, l'action à effectuer, et une consigne prête à adapter à votre projet. Les exemples partent d'un petit simulateur de devis ; adaptez-les librement à votre propre activité.

Ouvrez la bonne application. Ce guide concerne les agents en ligne de commande type Codex et Claude Code, dans une version qui propose ces commandes. L'application de bureau associée possède aussi son propre menu : tapez `/` pour vérifier ce qui est disponible dans votre conversation. Un chat web classique n'expose généralement pas ce même jeu de commandes ; ne supposez pas non plus qu'un chat standard reprend automatiquement les fonctions de son équivalent en ligne de commande.

00 • `/goal` : donner un objectif et une preuve de réussite

`/goal` fixe un objectif qui peut faire continuer le travail entre plusieurs tours d'échange. Certains outils conservent cet objectif dans la tâche elle-même ; d'autres utilisent un évaluateur qui juge ce que la conversation démontre, sans vérifier lui-même vos fichiers. Faites donc toujours apparaître les résultats de vos contrôles dans le fil de discussion.

« Améliore mon application » laisse trop de place à l'interprétation. « Le devis donne 270 après une remise de 10 % sur 300, et refuse une quantité négative » fournit au contraire une arrivée que vous pouvez vérifier objectivement.

Votre premier essai :

- Ouvrez un dossier de démonstration ou joignez le document utile. Vérifiez les outils disponibles.
- Choisissez un exemple et adaptez son périmètre. Si vous hésitez sur la version à produire, commencez par `/plan`.
- Envoyez l'objectif avec `/goal`. Observez les premiers contrôles avant de lui confier un travail plus long.
- À la fin, ouvrez le livrable et comparez le résultat aux critères annoncés au départ.

(Contrat de travail, exemple à adapter :) un simulateur de devis qui calcule juste. Preuve attendue : trois cas connus ($2 \times 150 = 300$; 300 avec 10 % de remise = 270 ; une quantité négative est refusée). Périmètre : une copie locale, des données fictives, aucune mise en ligne. La boucle attendue : agir → vérifier → corriger si nécessaire, jusqu'à un résultat vérifié, un blocage explicite, ou un arrêt décidé par vos propres contrôles.

PROMPT À COPIER

Prompt (Objectif : une application)

/goal Construire dans mon dossier de démonstration un simulateur de devis local avec quantité, prix unitaire et remise. Le résultat est terminé quand je peux saisir ces valeurs, lire le total et voir un message clair pour une valeur invalide.

Vérifie ces cas : quantité 2 et prix 150 donnent 300 ; avec 10 % de remise, le total devient 270 ; une quantité négative ou une remise supérieure à 100 % est refusée. Vérifie aussi le champ vide, le clavier et l'affichage sur téléphone. Affiche les résultats réels des contrôles.

Inspecte d'abord les fichiers disponibles et utilise la technologie déjà présente. Travaille seulement dans ce dossier, avec des données fictives. Préserve les fichiers sans rapport avec la demande. Ne publie rien et n'ajoute aucun paiement ou compte.

Après chaque modification, relance les contrôles concernés. Si un résultat échoue, cherche la cause avant de modifier autre chose. Si tu ne peux pas exécuter un contrôle, indique « non vérifié » et la raison. Arrête-toi avec le résultat vérifié ou un blocage précis, les essais effectués et la décision dont tu as besoin.

(Exemples pédagogiques rédigés pour ce guide, sans exécution réelle dans un modèle externe. Ils supposent un environnement capable de lire les pièces utiles et, pour l'application ou le fichier de données, de produire des fichiers.)

Gardez les commandes d'arrêt à portée de main, selon votre outil : consultation, mise en pause, reprise et suppression de l'objectif sont généralement disponibles sous des formes voisines de /goal, /goal pause, /goal resume et /goal clear en ligne de commande, ou via les boutons de suivi dans une application de bureau, mais les sous-commandes de pause et de reprise ne sont pas forcément transposables d'un outil à l'autre.

Un objectif peut s'arrêter sur une limite, une interruption ou un blocage. Il ne change pas automatiquement les permissions accordées à l'agent. La disponibilité de cette fonctionnalité dépend de la version de votre outil et, parfois, des réglages propres à votre organisation. Si votre menu la refuse, lisez la raison indiquée et consultez la documentation de votre environnement.

Pour borner un premier essai, choisissez un livrable petit et contrôlez vos limites d'usage. Une durée écrite dans le prompt reste une consigne, pas une minuterie garantie. Utilisez les contrôles natifs de l'application pour interrompre le travail si besoin.

Si vous travaillez dans un chat classique, reprenez le texte d'un objectif sans le préfixe /goal, joignez les éléments nécessaires, et demandez une première étape vérifiable. Vous devrez peut-être relancer la suite vous-même : écrire une consigne ne crée pas une fonction native de continuation.

01 • /plan : décider du chemin avant de construire

Vous voulez une application, mais le besoin tient encore en une phrase. Un plan vous aide à repérer une dépendance avant de consacrer une session entière à la mauvaise version.

À faire dans votre assistant : tapez `/plan`, puis décrivez le livrable. Lisez le plan proposé, précisez les choix manquants, et approuvez l'exécution seulement quand le périmètre vous paraît juste. Le fonctionnement précis varie selon l'outil.

PROMPT À COPIER

Prompt : `/plan`, consigne de travail

Prépare le plan d'un simulateur de devis destiné à une personne indépendante. Il doit calculer quantité × prix, puis une remise, et refuser les valeurs invalides.

Commence par examiner ce qui existe dans le dossier. Propose la plus petite version utile, les étapes et les contrôles attendus. Sépare les choix indispensables des améliorations futures. N'implémente rien avant mon accord sur ce plan.

Vous vérifiez : vous pouvez associer chaque étape à un résultat observable ; le plan comporte un contrôle final. *À garder en tête :* un plan approuvé ne prouve pas que l'application fonctionne ; il reste à la construire et à la tester réellement.

02 • /init : écrire les règles, une fois pour le projet

Vous répétez le ton, les limites ou la manière de tester à chaque nouvelle session. Un court fichier relu devient un point de départ commun à toutes les sessions suivantes.

À faire dans votre assistant : ouvrez le bon dossier, lancez `/init`, et relisez le fichier proposé. Gardez uniquement les règles qui s'appliquent vraiment à ce projet. Selon l'outil, le fichier généré porte un nom différent. Vérifiez lequel s'applique au vôtre.

PROMPT À COPIER

Prompt : `/init`, consigne de travail

Relis les instructions de ce projet et propose un ajout ciblé, sans effacer les règles existantes : interface en français, données fictives pour les démonstrations, vérification des calculs et du clavier avant livraison, aucune mise en ligne sans mon accord.

N'invente pas de commande de test : identifie celle du projet ou indique qu'elle manque. Signale les contradictions éventuelles avant de modifier le fichier.

Vous vérifiez : le fichier nomme le dossier autorisé, les vérifications, et les actions qui exigent votre accord. À *garder en tête* : n'y copiez aucun mot de passe ; une instruction durable trop vague se répète aussi durablement, sans jamais devenir plus précise.

03 • /model : choisir son moteur selon le travail

Une reformulation courte et un problème de calcul difficile n'appellent pas forcément le même niveau d'effort du modèle. Comparez toujours sur votre propre cas d'usage plutôt que sur une réputation générale.

À faire dans votre assistant : ouvrez le sélecteur de modèle et choisissez parmi ceux accessibles à votre compte. Pour comparer honnêtement, gardez exactement le même exercice et les mêmes critères d'un essai à l'autre. Les modèles, réglages et limites disponibles dépendent entièrement du produit et de votre propre accès.

PROMPT À COPIER

Prompt : /model, consigne de travail

Sur cet exercice de devis, propose une formule et vérifie trois cas : 2 articles à 150 sans remise ; les mêmes avec 10 % de remise ; une quantité négative. Explique brièvement les contrôles d'entrée nécessaires. Si une règle métier manque, formule ton hypothèse avant le calcul.

Vous vérifiez : le résultat résout les cas limites de votre exercice, avec un délai acceptable pour vous. À *garder en tête* : changer de modèle ne fournit ni le fichier manquant, ni un critère de réussite absent ; ce n'est jamais une solution à un flou de départ.

04 • /permissions : donner un périmètre, garder la main

Votre objectif nécessite de modifier des fichiers. Définissez où l'agent peut agir avant de le laisser enchaîner les étapes tout seul.

À faire dans votre assistant : dans le terminal, ouvrez le menu des permissions. Vérifiez les dossiers et les actions autorisés. Pour un essai, utilisez une copie locale du projet plutôt que l'original.

PROMPT À COPIER

Prompt : /permissions, consigne de travail

Avant d'exécuter ce projet, liste les fichiers et outils nécessaires, puis distingue les accès déjà disponibles de ceux qui manquent. Le périmètre demandé est le dossier de démonstration uniquement.

Tu peux préparer et vérifier les fichiers locaux autorisés. Publication, envoi, achat, suppression de l'original et modification d'un autre dossier exigent mon accord explicite. Si un accès manque, explique l'action précise qui en dépend.

Vous vérifiez : les accès accordés correspondent aux fichiers et outils réellement nécessaires ; votre prompt exprime aussi les limites du travail. *À garder en tête* : une phrase dans un prompt ne remplace pas un réglage technique. Ne désactivez jamais toutes les protections simplement pour supprimer une demande d'accès gênante.

05 • /mcp : vérifier les connexions avant de les supposer

L'agent vous parle d'un outil externe, mais vous ignorez s'il y a réellement accès. Le protocole MCP est un moyen de lui rendre des outils disponibles ; sa présence dans le menu ne garantit pas un accès effectif.

À faire dans votre assistant : ouvrez le menu des connexions et regardez ce qui est présent. Pour lire un fichier de données déjà joint, commencez directement avec le fichier : aucune connexion supplémentaire n'est nécessaire si vos outils savent déjà le lire.

PROMPT À COPIER

Prompt : /mcp, consigne de travail

Pour ce projet, vérifie quels outils sont réellement disponibles. J'ai besoin de lire [fichier ou ressource précise]. Commence par la méthode de lecture déjà accessible.

Si une connexion est nécessaire, explique laquelle et pourquoi. Ne l'installe pas et ne modifie pas ses autorisations sans mon accord. Distingue connexion configurée, accès confirmé et lecture effectivement réussie.

Vous vérifiez : un essai de lecture ciblé renvoie une donnée attendue, ou une erreur claire que vous pouvez résoudre. *À garder en tête* : n'installez jamais une connexion au hasard juste pour suivre un guide ; ne reliez qu'un service utile et connu.

06 • /status : lire les instruments, puis demander les preuves

La conversation dure et vous voulez savoir dans quel environnement vous travaillez. Séparez bien cette information technique de l'avancement réel du projet : l'une ne dit rien de l'autre.

À faire dans votre assistant : ouvrez le tableau de bord technique de la session. Pour l'objectif lui-même, consultez aussi son suivi dédié, puis demandez un bilan appuyé sur les résultats réellement obtenus.

PROMPT À COPIER

Prompt : /status, consigne de travail

Fais un point sur l'objectif en cours sans le modifier. Présente un tableau : critère de réussite, état (vérifié / à vérifier / bloqué), preuve disponible, prochaine action.

Cite les fichiers ou résultats réellement consultés. N'infère pas qu'un test passe parce que le code paraît correct. Termine par la seule décision qui nécessite mon intervention, s'il y en a une.

Vous vérifiez : le bilan relie chaque critère à un fichier, un calcul ou un test effectivement exécuté. *À garder en tête :* « j'ai presque terminé » ne permet pas d'approuver une livraison. Demandez toujours ce qui est vérifié et ce qui reste ouvert.

07 • /review : faire relire le travail avant la livraison

La démonstration fonctionne sur le cas prévu. Une relecture cherche ce qui pourrait casser ailleurs : un champ vide, un calcul incorrect, ou un comportement existant modifié sans le vouloir.

À faire dans votre assistant : sélectionnez les changements à relire et lancez la revue. Lisez les constats avant de demander des corrections. Selon l'outil, cette commande peut être un alias d'une fonctionnalité de revue de code intégrée ; les options exactes ne sont pas forcément interchangeables d'un outil à l'autre.

PROMPT À COPIER

Prompt : /review, consigne de travail

Relis uniquement les changements du simulateur de devis. Cherche les erreurs de calcul, les entrées invalides mal traitées et les régressions de clavier ou d'affichage.

Pour chaque constat, indique l'emplacement, un scénario de reproduction et la conséquence. Sépare les défauts établis des hypothèses. N'applique aucune correction et ne publie aucun commentaire externe à ce stade.

Vous vérifiez : chaque défaut signalé décrit un cas reproductible et sa conséquence ; les points sans preuve restent de simples pistes à explorer. *À garder en tête :* la revue n'est pas une certification et ne remplace pas les essais réels. Pour un document texte, demandez plutôt une relecture ordinaire avec des critères éditoriaux.

08 - /compact : alléger le contexte, garder les décisions

Plusieurs essais se sont accumulés dans la conversation. Avant de la condenser, rendez explicites les décisions et les résultats dont la prochaine étape aura besoin, sinon ils risquent de se perdre dans le résumé.

À faire dans votre assistant : envoyez la consigne ci-dessous, vérifiez la synthèse produite, puis lancez la commande de compression. Contrôlez ensuite que le critère de réussite est toujours bien compris par l'agent.

PROMPT À COPIER

Prompt : à envoyer avant /compact

Prépare une synthèse de reprise : objectif exact, contraintes, décisions validées, fichiers utiles, contrôles exécutés avec leurs résultats, essais abandonnés et raison, puis prochaine action.

Conserve les nombres et conditions de réussite sans les arrondir. Distingue ce qui existe dans un fichier de ce qui a seulement été proposé. Ne présente pas les contrôles non exécutés comme réussis.

Vous vérifiez : après la synthèse, l'agent restitue les contraintes et le prochain contrôle sans rouvrir une piste déjà invalidée. *À garder en tête :* un résumé peut perdre une nuance. Sauvegardez les décisions importantes dans un document du projet avant une longue reprise.

09 - /resume : retrouver le fil, vérifier ce qui a changé

Vous reprenez le projet le lendemain. Le fil contient l'historique, mais les fichiers ont pu évoluer entre-temps, avec ou sans vous.

À faire dans votre assistant : lancez la commande de reprise et sélectionnez la bonne session sauvegardée. Reprendre une conversation et reprendre un objectif en pause sont deux opérations différentes. Vérifiez l'état de l'objectif avant de le relancer.

PROMPT À COPIER

Prompt : /resume, consigne de travail

Reprends le contexte de cette conversation et inspecte l'état actuel du projet. Compare les fichiers disponibles au dernier bilan : changements intervenus, contrôles devenus obsolètes et travail restant.

Préserve les modifications d'autres personnes. Propose la prochaine action utile à partir de l'état réel. Si l'objectif précédent est terminé ou n'est plus pertinent, explique-le avant de relancer du travail.

Vous vérifiez : l'agent confronte l'ancien bilan aux fichiers présents et identifie la prochaine étape encore nécessaire. *À garder en tête :* ne lui demandez jamais de continuer aveuglément depuis un résumé ancien, surtout si une autre personne a travaillé sur le projet entre-temps.

10 • /clear : ouvrir un nouveau départ quand le sujet change

Le simulateur est fini et vous passez à un autre projet. Évitez d'emporter ses hypothèses dans une demande sans rapport avec lui.

À faire dans votre assistant : conservez d'abord un bilan utile de ce que vous quittez, puis démarrez une conversation au contexte vide. Cette commande retire aussi, selon l'outil, l'objectif actif en cours.

PROMPT À COPIER

Prompt : à envoyer avant /clear

Avant de changer de conversation, prépare une fiche de clôture : livrable et emplacement, critères vérifiés, limites connues et éventuelles suites. Ne prétends pas qu'une publication a eu lieu si seule une version locale existe.

Indique ce que je dois conserver pour reprendre ce travail. N'efface aucun fichier et ne relance pas l'objectif.

Vous vérifiez : le nouveau brief suffit à comprendre le projet sans dépendre de phrases comme « fais comme tout à l'heure ». *À garder en tête :* cette commande n'annule pas les modifications déjà faites sur vos fichiers. Pour revenir sur une modification, utilisez le mécanisme de versionnement de votre projet.

Un projet, une session

Prenez le simulateur de devis sur une copie de travail. Prévoyez un premier créneau de 30 minutes pour cadrer et observer, sans vous promettre que tout sera terminé dans ce délai.

- **Préparez.** Utilisez /plan pour définir la petite version, puis vérifiez les autorisations et le dossier. Lisez le plan avant d'engager l'exécution.
- **Lancez.** Adaptez l'objectif du guide et activez /goal. Gardez les calculs 300 et 270 comme points de contrôle.
- **Observez.** Comparez les preuves au résultat attendu. Si la quantité négative est acceptée, l'objectif n'est pas rempli : faites corriger ce cas précis.
- **Clôturez.** Relisez les changements avec /review, refaites les essais pertinents, et notez les limites restantes. Si vous manquez de temps, utilisez le contrôle d'arrêt de votre outil et conservez un bilan de reprise.

Votre contrôle avant livraison : j'ai nommé un livrable précis ; je sais quel contrôle prouve qu'il fonctionne ; le dossier et les outils autorisés sont définis ; je sais comment interrompre l'objectif dans mon application ; j'ai relu les preuves et les limites du résultat.

À retenir

Choisissez une arrivée que vous pouvez reconnaître. Vous pouvez déléguer le chemin, puis ouvrir le fichier, refaire le calcul ou essayer le formulaire vous-même. Cette vérification personnelle donne un sens réel au mot « terminé ».

Références

Fonctions vérifiées dans les documentations officielles à la mi-septembre 2026. Les menus évoluent selon la version, la surface et les accès. Les exemples, critères et enchaînements présentés sont des propositions de pratique, sans classement mesuré des utilisateurs ni résultat garanti.

Ressources citées : documentation officielle sur les objectifs persistants dans un agent en ligne de commande (critères de réussite, cycle de travail) ; documentation du menu et des contrôles d'objectif d'une application de bureau ; documentation des commandes en ligne de commande, avec la distinction par rapport à un chat web classique ; documentation sur l'évaluation, l'arrêt, les permissions et la disponibilité de la fonctionnalité d'objectif dans un agent de code ; répertoire de référence des commandes disponibles, avec leur syntaxe et leurs différences.